

数据类型和(算术以及关系) 运算

主要内容

- **数值类型、相关运算符和常用数学函数**
- 字符串和相关运算符

基本数据类型

- 数值类型：
 - 整数 (int)：没有小数部分的数值，任意大小
 - 浮点数 (float)：有理数的一个子集，有上下限，有精度限制
 - 布尔值 (bool)：只有True和False两个值
 - 复数 (complex)：包含实部和虚部，用j表示虚数单位。比如 $1 + 4j$
 - 如何判断某个对象obj是布尔、整数或浮点数等数值类型？
 - **isinstance(obj, (float, int))**
- NoneType: 只有None一个值
- 字符串(str)：一系列的字符

整数(int)

- 整数字面量(integer literal)
 - 十进制整数: 0、9、123
 - 整数0前面可填充0(如000), **非0十进制整数不允许前面填充0**
 - 十六进制(hex)整数: **0x或0X**开头, 后面为十六进制数字 (0-9, 以及abcdef, 也包括其大写形式ABCDEF) 如0x10、0xfa、0xabcdef
 - 八进制(octet)整数:以**0o或0O**开始后面为八进制数字 (0-7) , 如0o35、0o11
 - 二进制(binary)整数:以**0b或0B**开头, 后面为二进制数字(0, 1) 如0b101、0b10011
- **拓展:** python3.6之后, 支持在数字中间位置使用**单个下划线**作为分隔提高数字的可读性, 类似于数学上使用逗号作为千位分隔符
 - 单个下划线可以出现在中间任意位置, 但**不能出现在开头和结尾位置, 也不能使用多个连续的下划线**。比如 **4_758_000**, **0b_1001_0111**, **0x0800_FFA0**
- 负整数: 一元符号运算符 + 整数, 比如 -1, -0x10
- 整数类型int可以表示**任意大的数值, 只要内存允许**

```
>>> 9**100                # 9的100次方
2656139888758747693387813220357796268292334526533944959745749617390
92490901302182994384699044001
```

```
>>> 007
SyntaxError: invalid token
>>> 000
0
>>> 0x10
16
>>> 0xfa
250
>>> 0XABCDEF
11259375
>>> 0o35
29
>>> 0b100
4
>>> 4_758_000
4758000
>>> 0b1001_0111
151

>>> +1
1
>>> -0x10
-16
```

(有限精度) 浮点数 (float)

- 内置类型浮点数(float)为有理数的一个子集, 任意实数可近似为某个浮点数
- 浮点数字面量: 仅支持十进制
 - 小数表示: 3.14 10. .45
 - **科学计数法**: <整数或小数><E或e>整数
 - 15e-2 (=15*10⁻² = 0.15) 3.14e-10 1.0E100
- IEEE 754双精度(64bit double)浮点数, 内部采用二进制的归一化后的科学计数法表示
 - 符号 + 指数(exponent, 11bit, 全0和全1特殊含义) + 尾数(mantissa, 52bit)
 - 有上下限 (最大值和最小值), 具有有限的有效数字位数
 - 特殊的浮点数: inf和nan
 - float('inf'): inf, 无穷大
 - float('nan'): nan, 非数值(Not a Number)

1	11	52+1
sign	exponent	mantissa

$$sign * mantissa * 2^{exponent}$$

$$\text{规格化数: } 2^0 = 1 \leq mantissa < 2$$

```
>>> 3.14
3.14
>>> 10.
10.0
>>> .45
0.45

>>> 15e-2
0.15
>>> 31.4e-11
3.14e-10
>>> 1.0E100
1e+100

>>> float('inf')
inf
>>> -float('inf')
-inf
>>> float('nan')
nan
```

```
>>> import sys
>>> sys.float_info
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-308,
min_exp=-1021, min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)
```

(有限精度) 浮点数 (float)

- 浮点数内部采用二进制表示和运算，转换和计算时会有精度误差

- 浮点数内部表示的位数限制
- 十进制的有限小数转变为二进制时可能是无限循环小数!

比如: $0.1_{(10)} = 0.00011001100110011..._{(2)}$

- 避免采用 `==`, `!=` 比较浮点数是否相等

- `abs(x - y) < tolerance`
- `math` 模块的 `isclose` 函数可用于比较浮点数的相等

$$\text{relative tolerance} = \frac{|a - b|}{\max(|a|, |b|)}$$

f'{expr:.18}': expr 保留小数点后18位

```
>>> f'{0.1:.18}'
'0.10000000000000000006'
>>> f'{0.1+0.1:.18}'
'0.20000000000000000011'
>>> f'{0.2:.18}'
'0.20000000000000000011'
>>> f'{0.1+0.1+0.1:.18}'
'0.30000000000000000044'
>>> f'{0.3:.18}'
'0.29999999999999999989'
```

```
>>> 0.1
0.1
>>> 0.1+0.1
0.2
>>> 0.1+0.1+0.1
0.3000000000000000004
>>> 0.1 + 0.1 + 0.1 == 0.3
False
>>> abs(0.1 + 0.1 + 0.1 - 0.3) < 1e-9
True
>>> import math
>>> help(math.isclose)
...
isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)

>>> math.isclose(0.1 + 0.1 + 0.1, 0.3)
True
```

浮点数:Nan和inf (拓展的内容)

- 两个特殊的浮点数:
 - nan: Not a Number(非数值)
 - inf: infinity(无穷大)
 - 通过float内置函数得到
 - 通过math.nan和math.inf得到
- 比较运算:
 - inf和-inf表示正负无穷大
 - 正负无穷大可以与其他数值比较大小
 - 两个inf相等
 - nan与其他数值 (包括inf和nan)比较相等和小时, 结果都为False

```
>>> inf = float('inf')
>>> nan = float('nan')
>>> inf
inf
>>> nan
nan

>>> inf - inf
nan
>>> inf + float('inf')
inf

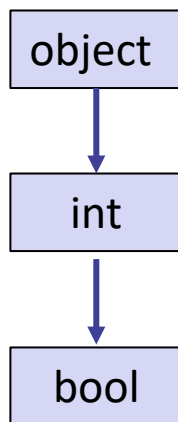
>>> inf == nan
False
>>> inf < nan
False
>>> inf > nan
False
>>> nan == nan
False

>>> nan is nan
True
```

布尔(bool) 和 NoneType

- 布尔(Boolean)类型bool:
 - 只有两个值, 分别为True和False
 - bool可以看成整数类型的特例
 - 在与整数等进行算术运算时, True和False分别对应整数1和0
- NoneType类型:
 - 只有一个值, 即None
 - 判断x是否为None建议采用 `x is None`, 不建议 `x == None`

```
>>> isinstance(True, bool)
True
>>> isinstance(True, int)
True
```



回顾: 判断是否为整数和浮点数等数值类型

```
>>> isinstance(2, (float, int))
True
>>> isinstance(True, (float, int))
True
>>> isinstance(3.14, (float, int))
True
```

复数(complex) (不要求)

复数(complex)类型:

- 一个复数包括实部 (real) 和虚部 (imaginary) 两个部分
- **虚数字面量**: 采用十进制, 浮点数或者整数, 后面为j (或者J), **中间不能有空格**

$3 + 4j$ $1.1 + 2.2j$ $3 + 4.2j$ $3-4.2j$ $4j$ $0j$

- 注意1j和j的区别: 1j表示虚部为1, 而j表示名字为j的变量
 - $4*j$ 表示4和变量j相乘
- 两个属性real和imag, 分别获得复数对象的实部和虚部

```
>>> (3-4j).real
3.0
>>> (3-4j).imag
-4.0
```

运算符

- 运算符：一种特殊符号（比如+），表示要对一个或者多个值（称为运算数）执行的运算
 - 只有1个运算数的运算符称为一元（单目）运算符
 - 有2个运算数的运算符称为二元（双目）运算符
 - 有3个运算数的运算符称为三元（三目）运算符
- python不支持其他语言（C/Java）等支持的**++（加1）和--（减1）**运算
- 算术运算符：进行算术运算的运算符

```
>>> -----1
1
>>> --1
1
>>> +---1
-1
```

运算符	含义	示例
+, -	一元符号运算	+8, -8
+, -	加法和减法	35 + 4, 2.5 + 3.5
*	乘法	(35 + 4) * 5, 3.4*5.2 + 4
/	真除法或浮点除法, 结果为浮点数	5 / 2 = 2.5 4 / 2 = 2.0 10 / 3.2 = 3.125
//	整除, 求整商, 结果可为整数或浮点数 - 两个整数整除, 结果为整数 - 运算数有浮点数时, 结果为浮点数	5 // 2 = 2 4 // 2 = 2 5.1 // 2.2 = 2.0
%	取余或求模。有浮点数时结果为浮点数 $a \% b = a - a // b * b$	5 % 2 = 1 4 % 2 = 0 5.5 % 2 = 1.5
**	幂运算	4**2 = 16 4**0.5 = 2.0

算术运算符： 优先级及混合运算

- 算术运算符支持两个不同的**数值类型**的对象之间的运算，即混合运算，比如浮点数+整数
 - 自动类型提升：针对数值类型的对象之间的算术等运算，将低级别的运算数提升为更高级别的类型后再进行运算
 - 如果其中包含complex对象，则其他数字类型转换为complex对象，结果为complex对象
 - 如果其中包含float对象，则其他数字类型转换为float对象，结果为float对象
 - bool可以看成整数类型的特例，True和False分别对应整数1和0

类型提升：bool → int → float → complex

- 有多个运算符时如何决定运算的顺序？
 - 算术运算符具有不同的优先级顺序，优先级高的优先计算
 - 对于同级的运算符，除了幂运算**为**右结合**运算外，其余同级为**左结合**运算
 - 可通过**小括号改变运算顺序**，小括号可嵌套

```
>>> True + 4 + 4.5
9.5

>>> 2 ** 3**2
512

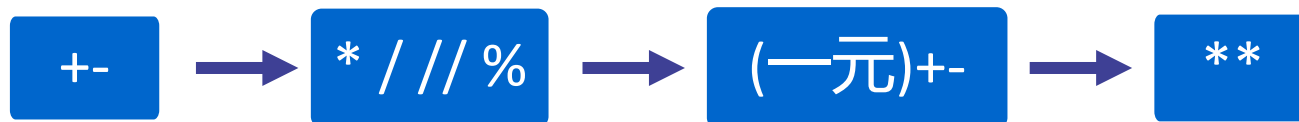
>>> (2**3) ** 2
64

>>> [2 * 3] * 2
[6, 6]
```

• 注意：中括号[]和大括号{}不用于数学计算，而是用于定义列表、字典以及集合等容器对象

PEMDAS规则

- Parentheses
- Exponents
- Multiplication and Division
- Addition and Subtraction



四组运算符优先级顺序：从低到高

除法类运算符

/	真除法或浮点除法, 结果为浮点数	$5 / 2 = 2.5$ $4 / 2 = 2.0$ $10 / 3.2 = 3.125$
//	整除, 求整商, 结果可为整数或浮点数 • 两个整数整除, 结果为整数 • 运算数有浮点数时, 结果为浮点数	$5 // 2 = 2$ $4 // 2 = 2$ $5.1 // 2.2 = 2.0$
%	取余或求模。有浮点数时结果为浮点数 $a \% b = a - a // b * b$	$5 \% 2 = 1$ $4 \% 2 = 0$ $5.5 \% 2 = 1.5$

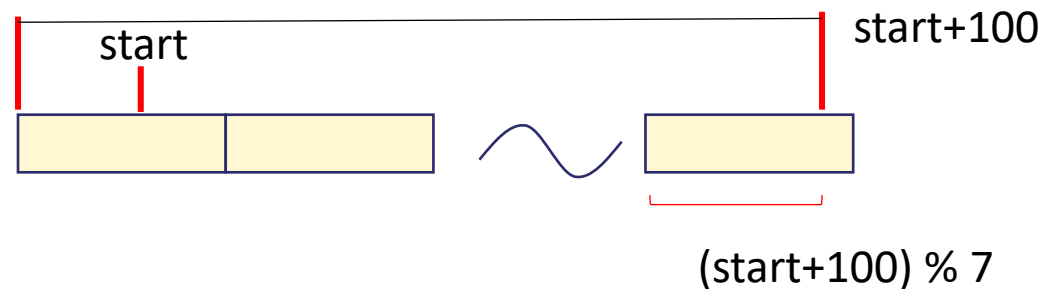
```
>>> 5 % 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer modulo by zero
```

- 除数不能为0, 否则抛出异常**ZeroDivisionError**。比如 $2 / 0$ $2 // 0$ $2 \% 0$
- 求模和整除运算符的应用
 - 获取个位数 $230857 \% 10 = 7$
 - 获取百位以上数字 $230857 // 100 = 2308$
 - 判断奇偶 $230857 \% 2 == 0$
 - 解决周期问题 $\text{start in } [0, 6], n\text{天之后星期几?}$

	m个数字	n个数字
--	------	------

$$x // (10 ** n) \% 10 ** m$$
$$x \% (10 ** (m + n)) // 10 ** n$$

```
>>> start = 3 # 今天是周四
# 100天之后星期几?
>>> (start + 100) \% 7 # 星期六
5
```



按位运算符(拓展内容)

运算数都要求为整数

- (不要求) 负数采用补码表示
- 移位运算符: 左移或右移n位
- 按位算术运算符, 考虑一个比特
 - &运算符:
 - $0 \& a = 0$ 对应位置为0
 - $1 \& a = a$ 得到对应位置的值
 - |运算符
 - $1 | a = 1$ 对应位置为1
 - $0 | a = a$ 得到对应位置的值
 - ^运算符: 半加, 不带进位的加法
 - $a \wedge a = 0$; $a \wedge b \wedge b = a$
 - $1 \wedge a = \sim a$ 对应位置取反
 - $0 \wedge a = a$ 对应位置不变

运算符	描述	示例
左移 $x \ll n$	x 左移 n 位, 右边补0, 等价于 $x * 2^n$	$5 \ll 2 = 20$
右移 $x \gg n$	x 右移 n 位, 右边补符号位, 等价于 $x // 2^n$	$21 \gg 2 = 5$
比特与 $a \& b$	a 和 b 相同位置的比特与(and)运算	$12 \& 10 = 8$
比特或 $a b$	a 和 b 相同位置的比特或(or)运算	$12 10 = 14$
比特异或 $a \wedge b$	a 和 b 相同位置的比特异或(xor)运算	$12 \wedge 10 = 6$
比特取反 $\sim a$	a 的每个比特取反(not), 等价于 $-x - 1$	$\sim 12 = -13$

a	b	$a \& b$
1	1	1
1	0	0
0	1	0
0	0	0

a	b	$a b$
1	1	1
1	0	1
0	1	1
0	0	0

a	b	$a \wedge b$
1	1	0
1	0	1
0	1	1
0	0	0

a	$\sim a$
1	0
0	1

3	2	1	0
1	1	0	0
1	0	1	0

$12 = 2^3 + 2^2 = 0b1100 = 1 \ll 3 | 1 \ll 2$
 $10 = 0b1010$

按位运算符(拓展内容)

- $0 \& a = 0$ 对应位置为0
- $1 \& a = a$ 得到对应位置的值
- $1 | a = 1$ 对应位置为1
- $0 | a = a$ 得到对应位置的值
- $1 \wedge a = \sim a$ 对应位置取反
- $0 \wedge a = a$ 对应位置不变

2	1	0
r	w	x

```
EXEC  = 1 << 0    # 001
WRITE = 1 << 1    # 010
READ  = 1 << 2    # 100
```

`perm = READ | WRITE # 110, 表示读写权限`

```
def set_bit(n, pos):
    """设置第pos位为1 (从0开始计数), 其他不变"""
    return n | (1 << pos)

def clear_bit(n, pos):
    """清除第pos位 (设置为0), 其他不变"""
    return n & ~(1 << pos)

def toggle_bit(n, pos):
    """切换第pos位 (0变1, 1变0), 其他不变"""
    return n ^ (1 << pos)

def get_bit(n, pos):
    """获取第pos位的值"""
    return (n >> pos) & 1

if __name__ == "__main__":
    perm = 0 # 初始权限为0 (000)
    perm = set_bit(perm, 2) # 设置读权限
    perm = set_bit(perm, 1) # 设置写权限
    print(f"权限位: {perm:03b} (读写权限)")
    perm = toggle_bit(perm, 0) # 切换执行权限
    perm = clear_bit(perm, 1) # 清除写权限
    print(f"读权限: {get_bit(perm, 2)}")
```

bitwise.py

比较(关系) 运算符

注意: == != <= >= 运算符中要求之间**不能有空格**

- 比较运算符运算的结果为True或False (bool类型)
- 对象的值是否相等: 两个对象可以是不同类型
 - $x == y$, $x \neq y$: 等于、不等于
 - 要判断x是否为None, 建议使用 $x \text{ is None}$ 而不是 $x == \text{None}$
- 两个对象是否同一个对象: 两个对象可以是不同类型
 - $x \text{ is } y$, $x \text{ is not } y$: 等价于判断 $\text{id}(x) == \text{id}(y)$, $\text{id}(x) \neq \text{id}(y)$
 - 不建议 $\text{not } x \text{ is } y$
- 判断数值类型的对象的值的大小:
 - 对象都是数值类型(不包括复数, 复数间无法比较大小) 时可以比较
 - $x < y$, $x \leq y$, $x > y$, $x \geq y$: 小于、小于等于、大于、大于等于
- **比较运算符**可以连用, 连用时**不考虑传递性**, 仅仅**相邻数之间的比较**
 - $x < y < z$ 表示 $x < y$ 且 $y < z$
 - $x < y > z$ 表示 $x < y$ 且 $y > z$
 - $x > y \text{ is } z$ 表示 $x > y$ 且 $y \text{ is } z$, 即 $x > y$ and $y \text{ is } z$

- **先算后比**: 算术运算符的优先级相比比较运算符优先级更高
- 所有比较运算符的优先级相同

```
>>> x, y, z = 12, 18, 13
```

```
>>> x < y, x <= y, x > y, x >= y    # (True, True, False, False)
```

```
>>> x < y < z, x < y > z    # (False, True)
```

```
>>> 4**2 + 8**2 < 5**2 + 7**2
False
```

内置数学函数

函数	含义	实例	结果
abs(x)	x的绝对值。 如果x为复数，返回复数的模	abs(-1.2) abs(1-2j)	1.2 2.23606797749979
divmod(a,b)	返回a除以b的商和余数	divmod(5, 3)	(1,2) <code>>>> x, y = divmod(5, 3)</code>
pow(x,y[,z])	返回x**y。如果传递了z，则为pow(x, y) % z	pow(2, 10) pow(2, 10, 10)	1024 4
max(arg1,arg2,*args)	取最大值	max(1, 7, 3, 15, 14)	15
min(arg1,arg2,*args)	取最小值	min(1, 7, 3, 15, 14)	1
sum(iterable[,start])	求和，如果传递了start，表示加上start	sum((1, 2, 3)) sum((1, 2, 3), 44)	6 50
round(number[,ndigits])	取整到最近的整数。第2个参数ndigits给出了保留到多少位小数。ndigits可以为负数，表示保留10的多少次方	round(3.14159) round(3.14159, 4) round(314159, -3)	3 3.1416 314000

$$\text{round}(\text{numbers}, \text{ndigits}) = \text{round}(\text{number} \times 10^{\text{ndigits}})/10^{\text{ndigits}}$$

- round函数的取整规则并不是传统四舍五入(Round-Half-Up)
- **【拓展内容】** round采用银行家取整(Banker's Rounding, Round-Half-Even), 也称为四舍六入五成双
 - 取整到最近的整数(Round Nearest)
 - 如果正好为两个相邻整数的正中间时, 则取整到最近的偶数
 - 传统四舍五入中, 多个值四舍五入后再进行运算, 正好在两个相邻整数的中间的值总是往上取整, 会导致最终的结果偏大
 - 银行家取整中, 那些正好在两个相邻整数的中间的值以相等的概率进行舍或入, 可得到一个无偏的结果

```
>>> round(2.5)
2
>>> round(3.5)
4
>>> round(1.875, 2)
1.88
>>> round(1.625, 2)
1.62
```

```
>>> round(2.675, 2)
2.67
```

```
>>> from decimal import Decimal # 十进制高精度(缺省28) 浮点数
>>> Decimal(2.675)
Decimal('2.674999999999999982236431605997495353221893310546875')
```

为什么有时候不满足Round-Half-Even?

- 浮点数内部的计算和表示采用二进制

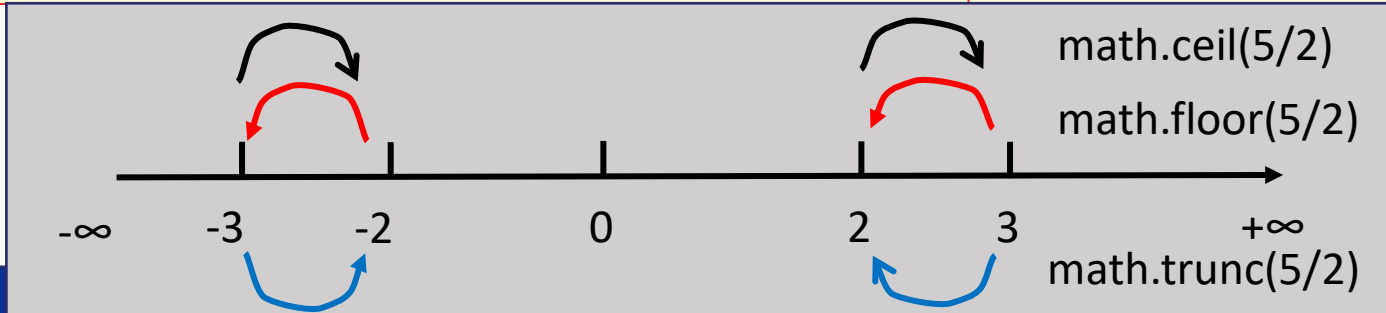
math模块

- 数学常量：
 - math.pi: 数学常量 π
 - math.e: 数学常量 e
 - math.nan: Not a Number
 - math.inf: infinity
- 特殊浮点数相关：
 - isfinite, isinf, isnan
 - isclose(a, b)
- 平方根、取整相关的函数、指数和对数函数
- 三角函数、双曲函数
- 排列组合函数等
 - perm(n, k=None): n中取k个排列
 - comb(n, k): n中取k个组合
 - gcd(...), lcm(...) 最大公约数和最小公倍数

```
>>> import math
>>> math
<module 'math' (built-in)>
>>> dir(math)
['__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'acos', 'acosh',
 'asin', 'asinh', 'atan', 'atan2', 'atanh',
 'ceil', 'comb', 'copysign', 'cos', 'cosh',
 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',
 'expm1', 'fabs', 'factorial', 'floor', 'fmod',
 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf',
 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt',
 'lcm', 'ldexp', 'lgamma', 'log', 'log10',
 'log1p', 'log2', 'modf', 'nan', 'nextafter',
 'perm', 'pi', 'pow', 'prod', 'radians',
 'remainder', 'sin', 'sinh', 'sqrt', 'tan',
 'tanh', 'tau', 'trunc', 'ulp']
>>> help(math)      # 查看math模块帮助
>>> help(math.sin)   # 查看math模块中的sin函数帮助
>>> math.pi
3.141592653589793
>>> math.e
2.718281828459045
```

math模块

函数	含义	实例	结果
<code>sqrt(x)</code>	$x(≥0)$ 的平方根	<code>sqrt(2)</code>	1.4142135623730951
<code>pow(x, y)</code>	$x ** y$, 返回浮点数	<code>pow(2, 3)</code>	8.0
<code>exp(x)</code>	$e ** x$	<code>exp(5)</code>	148.4131591025766
<code>log(x[,base])</code>	以base为底x的对数, base没有则为以e为底的自然对数, 即缺省为 $\ln(x)$	<code>log(math.e ** 2)</code> <code>log(4, 2)</code>	2.0 2.0
<code>log10/log2(x)</code>	以10或2为底的对数	<code>log10(100)</code> , <code>log2(16)</code>	2.0, 4.0
<code>factorial(x)</code>	整数 $x(≥0)$ 的阶乘, $x!$, $0!$ 为1	<code>factorial(5)</code>	120
<code>fabs(x)</code>	绝对值, 返回float	<code>fabs(-3)</code>	3.0
<code>fsum(seq)</code>	多个浮点数求和	<code>fsum([0.1]*10)</code>	1.0
<code>ceil(x)</code>	大于等于x的最小的整数(正无穷方向取整)	<code>ceil(1.2)</code> , <code>ceil(-1.6)</code>	(2, -1)
<code>floor(x)</code>	小于等于x的最大的整数(负无穷方向取整)	<code>floor(1.8)</code> , <code>floor(-2.1)</code>	(1, -3)
<code>trunc(x)</code>	截取为最接近0的整数 (原点方向取整)	<code>trunc(1.2)</code> , <code>trunc(-2.8)</code>	(1, -2)



【拓展内容】：`//`运算

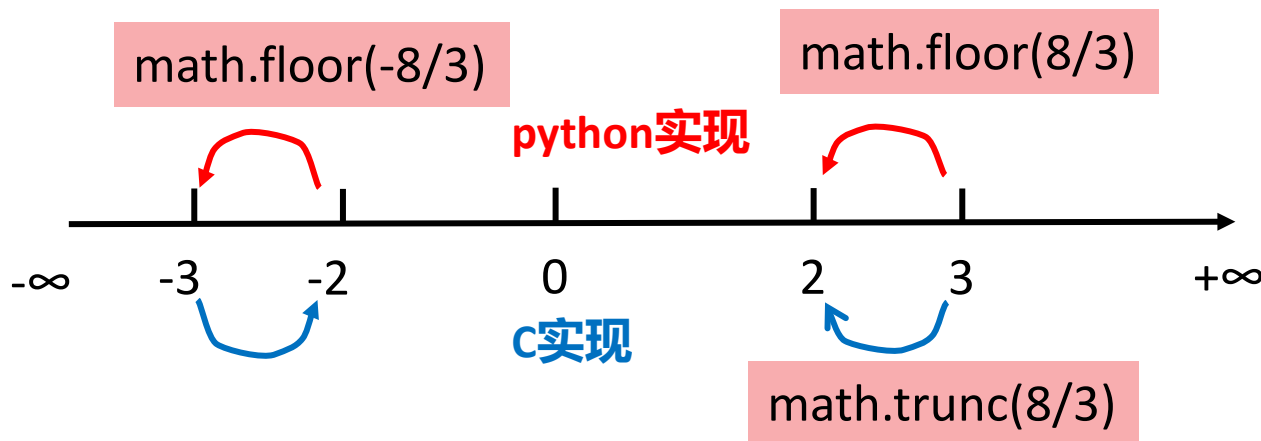
- 如果整除和求模运算时，其结果为负数时如何计算？负数的情形比较少见

- 整除（整商）运算：

- Python的实现为向负无穷方向舍入（`floor()`函数），即`math.floor(a / b)`
- C/C++、Java为向原点(0)方向舍入（`trunc()`函数）
- 对于结果为正数而言，两种实现的结果一致

- 求模运算： $a \% b == a - a // b * b$

```
>>> -8 // 3
-3
>>> -8 % 3
1
```



主要内容

- 数值类型和相关运算符
- **字符串和相关运算符 (2.5, 3.1.2)**

字符串(str)

- 字符串表示一串字符组成的文本，通过**字符串界定符**定义，以某个字符串界定符开始，以同样的字符串界定符结束
 - 其他语言一般用双引号定义字符串字面量。Python的字符串界定符可以使用**单引号**和**双引号**
 - 交互模式中输出字符串的内部表示时采用单引号定义
 - 如果字符串中有双引号，则可使用单引号定义，反之亦然
- `'abc'` `"Python程序设计基础"` `"What's your name?"` `"'Thank you!' she said."`
- 字符串可以为**空字符串**，即不包含任何字符，可用两个连续的界定符定义：比如 `' '` 或 `''''`
 - 两个以上连续（中间可用空格等分隔）的**字符串字面量**被合并为一个字符串 `'12' '34'` 相当于 `'1234'`
 - 内置函数**len(s)**：s为字符串时，返回字符串中的字符个数
 - 字符串字面量定义的限制：
 - 短字符串定义：单引号和双引号定义的字符串，**不能跨越多行**
 - 字符串中的文本如果出现了字符串界定符本身怎么办？见下一页slide

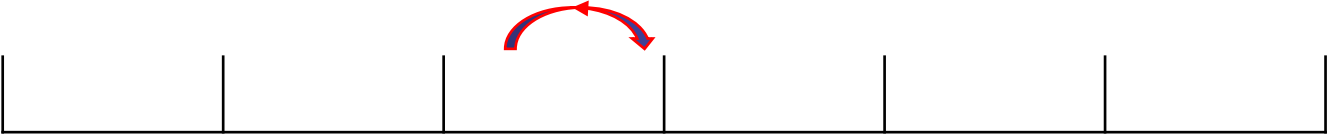
```
>>> "abc"
'abc'
>>> len('')
0
>>> len('12'
      "34")
4
```

```
# 下列字符串定义会报错 SyntaxError: EOL while scanning string literal
s1 = 'a line with no matching delimiter
s2 = ' be careful of Chinese punctuation '
```

编程经常出现的问题是采用了全角字符，包括中文标点符号

字符串转义

制表位



- 如果一个字符串内容中出现界定符(如单双引号)本身，可在引号前加上**转义字符**表示其不是字符串界定符，而是普通的引号
 - `\'` 表示单引号本身， `\"`表示双引号本身
 - `\\` 表示单个的反斜杠
- 还可通过转义序列描述一些控制字符

转义序列	含义	转义序列	含义
<code>\'</code>	单引号	<code>\"</code>	双引号
<code>\\</code>	反斜杠	<code>\t</code>	制表符Tab
<code>\n</code>	换行符 Newline	<code>\r</code>	回车CarriageReturn， 移动到当前行的开始
<code>\a</code>	响铃BEL	<code>\b</code>	后退Backspace
<code>\f</code>	FormFeed， 输出到打印机时换页	<code>\v</code>	垂直制表VT

```
>>> print('\hello\' and "hello")
'hello' and "hello"
>>> print('name\tphone\taddr\nidle\t123344\tidle\'house')
name    phone   addr
idle    123344  idle'house
```

```
>>> print("\\hello\nhow\tare \"you\"?\\")
\hello
how    are "you"?\\
```

长字符串

- 短字符串定义：单引号和双引号定义的字符串不能跨越多行
 - 通过\n表示换行符，从而定义多行字符串
- 长字符串：
 - 采用**三(单/双)引号**'''或'''作为字符串界定符
 - 长字符串可以跨越多行
 - 除了可跨越多行外，其他与短字符串完全一样，也支持字符转义

- python语言支持一般采用#进行单行注释，#之后同一行的内容忽略掉
- python语言本身不像C等支持多行注释，但是由于表达式执行后没有任何side effect，可以采用长字符串来支持**多行注释**

```
>>> print('''first line
line 2
line 3
last line.''' )
first line
line 2
line 3
last line.
```

```
>>> print('first line \nline 2\nline 3\nlast line.')
first line
line 2
line 3
last line.
>>> print('''first line \nline 2
line 3
last line.''' )
first line
line 2
line 3
last line.
```

原始字符串

- 短字符串和长字符串的内容中如果有许多字符需要进行转义会比较繁琐

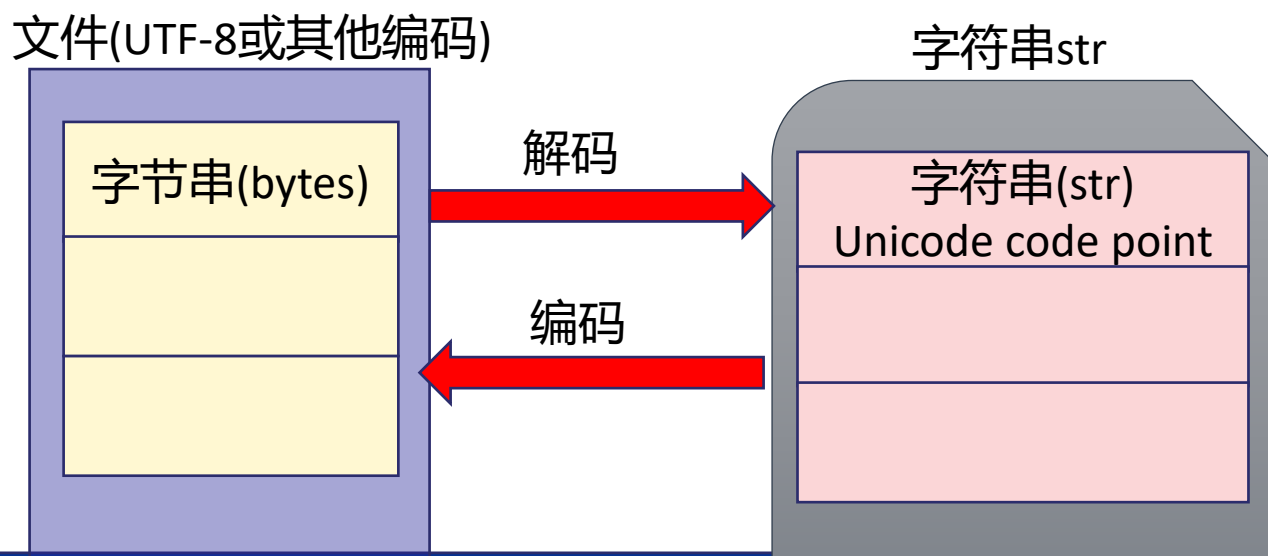
```
>>> print(''要让变量a指向字符串"C:\\Program Files (x86)\\Python35\\bin"
应该这样定义: a = "C:\\\\Program Files (x86)\\\\Python35\\\\bin" '')
要让变量a指向字符串"C:\\Program Files (x86)\\Python35\\bin"
应该这样定义: a = "C:\\Program Files (x86)\\Python35\\bin"
```

- 原始(raw)字符串定义:
 - 字符串界定符前面**加字母r或R**，界定符之间的所有内容作为原始字符串的内容
 - 原始字符串定义中，特殊字符不需要转义

```
>>> print(r''要让变量a指向字符串"C:\\Program Files (x86)\\Python35\\bin"
应该这样定义: a = "C:\\Program Files (x86)\\Python35\\bin" '')
要让变量a指向字符串"C:\\Program Files (x86)\\Python35\\bin"
应该这样定义: a = "C:\\Program Files (x86)\\Python35\\bin"
```

字符集

- 字符集(Charset): 系统支持的多个抽象字符的集合, 每个字符在该字符集中可以通过一个称为码点(Code Point)的数字唯一标识
- 字符编码(Encoding): 字符集中的字符在计算机内部存储时如何表示, 可映射为1个字节或者多个字节 (称为Code Unit)
- 我们通过IDLE等编写的源文件(.py), 缺省采用unicode字符集, 编码方式为UTF-8编码
- 以字节方式保存在外部存储(硬盘/U盘等)
 - 编辑器在加载文件时, 基于UTF-8编码, 将外部存储的字节流解释为Unicode字符集中的字符串
 - 编辑器在保存文件时, 基于UTF-8编码, 将字符串编码成字节流后保存到外部存储



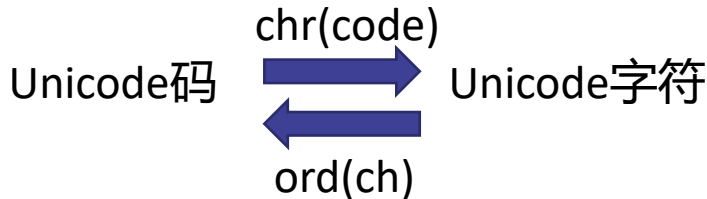
常见字符集

- ASCII或US-ASCII: American Standard Code for Information Interchange
 - 128个字符组成:
 - 前面32个字符为控制字符
 - 后面96个字符: `!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~\x7f`
 - 其CodePoint (也称为ASCII码) 就是字符集中的序号 (0-127), 编码时采用1个字节保存字符的ASCII码
- ISO-8859-1 (Latin-1): 西欧采用的 8 位字符集, 共256个字符。编码采用1个字节保存字符序号
 - 前面128个字符对应着ASCII字符集中的字符
 - 后面的128个字符 (128-255): 包括了大部分使用变音符号的拉丁字母语言中的字符
- GB2312: 中国国家标准简体中文字符集, 可编码成1或2个字节
 - 最高位为0时, 1个字节对应着ASCII字符集中的字符
 - 中文使用2个字节表示, 每个字节的最高位都为1
- GBK (cp936): GB2312的扩充, 支持更多的汉字, 包括繁体汉字等
- GB18030 (cp54936): 加入少数民族字符

前面128个字符
(ASCII字符集的字符)

GB2312、Unicode字符集等

常见字符集: Unicode



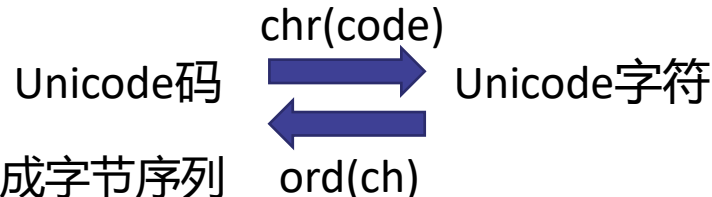
Unicode(统一码): <http://www.unicode.org/>

- Unicode统一了全球不同语言采用的字符集, 通过Unicode Code Point(Unicode码)描述
- ASCII码(0x00-0x7f)和Latin-1(0x80-0xff)是Unicode字符集的子集
- unicodedata模块给出了访问unicode数据库的一些方法, 包括所属的category、name、字符宽度(east_asian_width) 等
- **内置函数ord返回字符对应的unicode。已知unicode, 可使用内置函数chr返回对应的字符**
- 字符串字面量定义中可以通过\ddd、\xhh、\uxxxx或\Uxxxxxxxx等描述相应码点对应的字符
 - 通过字符名字描述 \N{<name>}

```
>>> print('八进制(\141) 十六进制(\xf7) 十六进制Unicode(\u7a0b)以及(\U00005e8f)')
八进制(a) 十六进制(÷) 十六进制Unicode(程)以及(序)
>>> print("\N{rightwards arrow} \N{greek small letter alpha}")
→ α
```

\ddd	1-3位八进制数(ddd)对应的字符	\xhh	2位十六进制数(hh)对应的字符
\uxxxx	4位十六进制数对应的UNICODE字符	\Uxxxxxxxx	8位十六进制数对应的UNICODE字符

内置函数 chr和ord

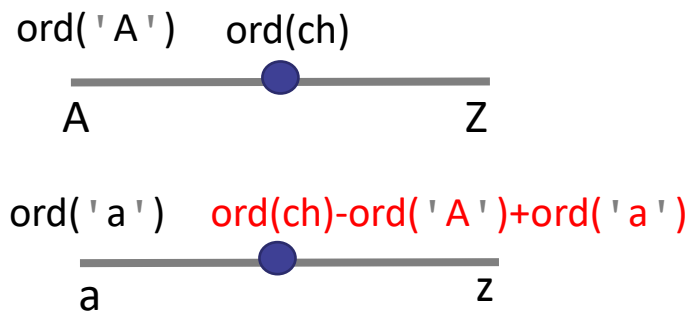


- Unicode字符可采用UTF-8, UTF-16, UTF-32进行编码, 将unicode对应的字符转换成字节序列
- UTF-8**: Unicode transformation format, Code page: 65001
 - 目前使用最为广泛的Unicode编码方式。Python3的字符串采用UTF-8编码
 - 编码成1~6字节。ASCII字符占1个字节, 其编码与ASCII编码一致。中文字符编码成3个字节
- Unicode字符集中最前面128个字符为ASCII字符集中的字符:
 - 前面32个字符为控制字符
 - 后面96个字符: `!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~\x7f`
- 内置函数ord返回字符对应的unicode。已知unicode, 可使用内置函数chr返回对应的字符**

```
>>> ord('\t')    >>> ord('A')    >>> chr(0x03b1)
9                65                'α'
>>> ord('\n')    >>> ord('a')    >>> chr(0x03b2)
10               97                'β'
>>> ord('0')     >>> chr(48)     >>> chr(0x2200)
48               '0'                '∀'
```

```
ch = 'M'
# ch_lower = ch.lower()
ch_lower = chr(ord(ch) - ord('A') + ord('a'))
print(ch_lower)
```

英文字母的大小写转换



字符串转义：【拓展内容】非法转义序列

\表示**后面的字符可能不是原有的含义**，需要特别对待，即可能具有特殊含义

- 如果后面的字符的确有特殊的含义，即是合法的转义序列，则转义字符移走，使用特殊的含义
 - \后面是单引号、双引号和反斜杠时，表示后面的字符不用于字符串界定和转义，而是字符本身
 - \后面是n/t/r/a/b/v/f时，表示对应的控制字符（换行、制表等）
 - \后面跟着数字或x/u/U/N等表示根据码点描述的相应字符
- **【拓展内容】**：如果后面的字符并没有特殊的含义，则**转义字符仍然保留**，比如 '\c'包含两个字符：反斜杠和c

```
>>> print('d:\dlmao\python')
d:\dlmao\python
>>> print('d:\tony\network')
d:      ony
etwork
>>> print('d:\\dlmao\\python')
d:\dlmao\python
>>> print('d:\\tony\\network')
d:\tony\network
```

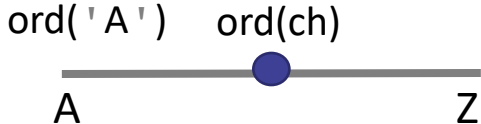
```
# 开启python开发模式时会显示warning
C:\ python -X dev
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34)
[MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> '\c'
<stdin>:1: DeprecationWarning: invalid escape sequence '\c'
'\\c'
>>>
```

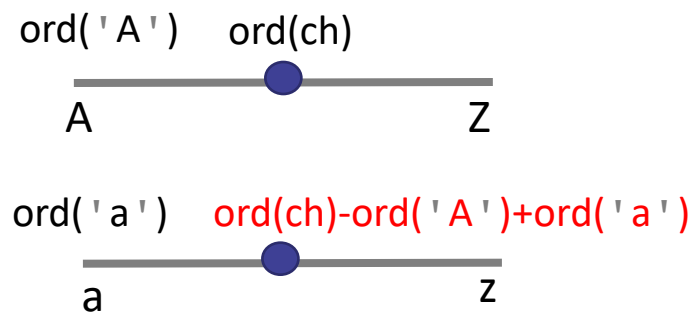
字符串(有序序列)的下标访问

Diagram illustrating the string `abcde` with indices `0` to `4` above the characters and `-5` to `-1` below the characters. The string length is `len(s)=5`. The range `[0,5)` is indicated above the string, and the range `[-5,0)` is indicated below the string.

s='abcde'

字符串是不可变的有序序列

- 通过下标访问有序对象中对应位置的元素，即**对应位置的字符组成的字符串**：`s[index]`
 - 第一个元素下标为0，第二个元素下标为1，以此类推，直到`len(s)-1`
 - 最后一个元素下标为**-1**，倒数第二个元素下标为**-2**，以此类推，直到**`-len(s)`**
 - 负数下标 `-i` 对应于 `len(s)-i`
 - 合法下标为 `[-len(s), len(s)-1]`
 - 如果下标越界，程序会报错(Exception `IndexError`)
- 
- The diagram shows a horizontal line representing the range of ASCII characters from 'A' to 'Z'. The character 'A' is at the left end and 'Z' is at the right end. A blue dot is positioned on the line, representing a character 'ch'. Above the dot is the label 'ord(ch)', and above the 'A' is the label 'ord('A')'.



```
>>> hexdigits = '0123456789abcdef'
>>> hexdigits[0]
'0'
>>> hexdigits[15]
'f'
>>> hexdigits[-1]
'f'
>>> hexdigits[16]
...
IndexError: string index out of range
```

```
ch = 'M'
# ch_lower = ch.lower()
ch_lower = chr(ord(ch) - ord('A') + ord('a'))
print(ch_lower)
# 另外一种实现大写变小写的方式？
• 提示：不使用chr，而是使用下标
```

字符串(有序序列)的下标访问

0	1	2	3	4	[0,5)
a	b	c	d	e	len(s)=5
-5	-4	-3	-2	-1	[-5,0)

- 通过下标访问有序对象中对应位置的元素，即**对应位置的字符组成的字符串**：`s[index]` s='abcde'
- `s.index(sub)` 检查sub是否在s出现。如果出现，返回其**首次出现**时其开始位置的下标。否则抛出异常**ValueError**
- `s.count(sub)` 返回子串sub出现的次数，找到子串后，**在之后的下一个位置**继续...

```
>>> s = "apple,peach,banana,peach,pear"
>>> s.index('p')
1
>>> s.index('z')
Traceback (most recent call last):
  File "<python-input-128>", line 1, in <module>
    s.index('z')
    ~~~~~^~~~~
ValueError: substring not found
>>> s.index('peach')
6
>>> s.count('peach')
2
>>> 'pppppp'.count('pp')
3
>>> 'pppppp'.count('pp')
3
```

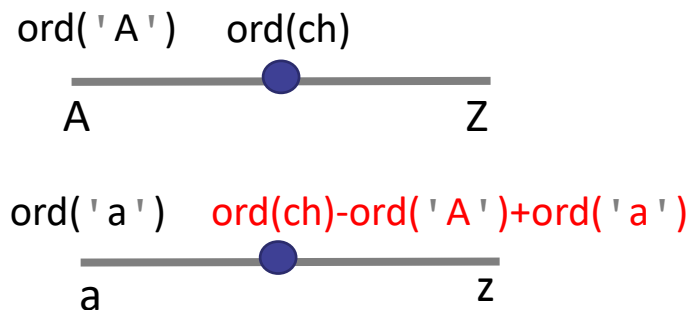
字符串(有序序列)的下标访问

自己实现十六进制数字与整数之间的转换

hex_digit_int.py

```
def hex_digit_to_int(ch):
    if '0' <= ch <= '9':
        return ord(ch) - ord('0')
    elif 'a' <= ch <= 'f':
        return ord(ch) - ord('a') + 10
    elif 'A' <= ch <= 'F':
        return ord(ch) - ord('A') + 10
    else:
        raise ValueError(ch)

def hex_digit_to_int2(ch):
    HEX_CHARS = "0123456789abcdef"
    try:
        return HEX_CHARS.index(ch.lower())
    except ValueError:
        raise ValueError(ch)
```



```
def int_to_hex_digit(n):
    if 0 <= n <= 9:
        return chr(n + ord('0'))
    elif 10 <= n <= 15:
        return chr(n - 10 + ord('a'))
    else:
        raise ValueError(n)

def int_to_hex_digit2(n):
    HEX_CHARS = "0123456789abcdef"
    if 0 <= n < len(HEX_CHARS):
        return HEX_CHARS[n]
    else:
        raise ValueError(n)
```

返回某种类型的实例对象：构造函数法

int/float/bool/complex/str等内置**类对象**，分别对应整数、浮点数、布尔、复数和字符串等类型

- 这些类对象**可以像函数一样调用**，称为构造函数，返回一个该类型的实例对象

int() 返回整数0	int(x) x为数值类型(不包括复数) True或False转换为1或0 浮点数取(原点截取)整数部分	int(x, base=10) x为字符串, base缺省为10, x为指定进制(缺省十进制)的整数字符串(忽略首尾空格)
float() 返回浮点数0.0	float(x) x为数值类型(不包括复数) True或False转换为1.0或0.0	float(x) x为字符串(忽略首尾空格) 仅支持十进制, 将十进制的整数字符串或浮点数字符串转换为浮点数
bool() 返回False	bool(x), x进行真值判断 (介绍分支结构时展开) 真值判断: x为值为0的数值对象, 长度为0的空容器对象(如空字符串)、None、False时返回False, 否则返回True	
str()返回空字符串	str(obj) 将对象转换为字符串	

```
int(), int(3.14), int('314'), int('ff',16)           # (0, 3, 314, 255)
float(), float(3), float('-3'), float('3.14'), float('10e2') # (0.0, 3.0, -3.0, 3.14, 1000.0)
bool(), bool(3), bool(0), bool(3.14), bool(bool)      # (False, True, False, True, True)
str(), str(3.14), 'abc' + str(4)                     # ('', '3.14', 'abc4')
```

```
>>> int(' 0x45 ', 16)
69
>>> int(' 45 ', 16)
69
>>> float(' 1e-2 ')
0.01
```

内置函数str、repr、ascii

str(obj)	返回一个对人友好的printable字符串，让 用户了解该对象 的相关信息。print函数的参数可为任意对象obj，相当于print(str(obj))
repr(obj)	拓展： 返回描述对象的内部表示(representative)的字符串，让 程序员了解该对象 。该字符串存储的一般是可以写在程序中的字面量。制表、回车、换行字符以\t\r\n表示，其他ASCII控制字符以\x...形式描述。eval(repr(obj))一般可以得到obj
ascii(obj)	拓展： 类似于repr，只是返回的字符串中仅仅允许ASCII字符集中的字符

对象(obj)	print(str(obj))	print(repr(obj))
3	3	3
True	True	True
[1, 2]	[1, 2]	[1, 2]
'123'	123	'123'
r'1\2\\3'	1\2\\3	'1\\2\\\\3'

```
>>> text= r'c:\tmp\python 程序设计'
>>> print(text) # print(str(text))
c:\tmp\python 程序设计
>>> text
'c:\\tmp\\python 程序设计'
>>> print(repr(text))
'c:\\tmp\\python 程序设计'
>>> print(ascii(text))
'c:\\tmp\\python \u7a0b\u5e8f\u8bbe\u8ba1'
```

- **交互式console**：如果执行的语句为表达式obj，且其值**不为None**时调用**print(repr(obj))**。
对于字符串而言，输出结果为**采用转义方式描述的字符串字面量**

字符串转换

函数	含义	实例	结果
<code>bin(x)</code>	把整数x转换为二进制字符串，包含0b前缀	<code>bin(4)</code>	<code>'0b100'</code>
<code>oct(x)</code>	把整数x转换为八进制字符串，包含0o前缀	<code>oct(25)</code>	<code>'0o31'</code>
<code>hex(x)</code>	把整数x转换为十六进制字符串，包含0x前缀	<code>hex(75)</code>	<code>'0x4b'</code>
<code>str(obj)</code>	把对象obj转换为字符串	<code>str(16), str('程序')</code>	<code>('16', '程序')</code>
<code>repr(obj)</code>	把对象obj转换为内部表示的字符串	<code>repr(16), repr('程序')</code>	<code>('16', "'程序'")</code>
<code>ascii(obj)</code>	把对象obj转换为内部表示的ascii字符串	<code>ascii('程序')</code>	<code>'"\u7a0b\u5e8f"'</code>

问题：用户输入一个三位自然数，计算并输出其百位、十位和个位上的数字。

```
x = int(input('请输入一个三位数: '))
hundreds = x // 100
tens = x // 10 % 10
ones = x % 10
print(hundreds, tens, ones)
```

```
text = input()
x = int(text)
```

内置函数eval和exec(2.2.3)

eval(source, globals=None, locals=None)

- **执行source中的代码**，source一般为字符串，**包含的代码必须为表达式**。该表达式在globals和locals指出的环境（**缺省为当前环境**）下执行，**表达式求解后的对象作为返回值**。如果指定了globals和locals，则只会查找指定名字空间的名字

exec(source, globals=None, locals=None)

- 与eval类似，执行source对应的代码，返回None

```
text = input('请输入一个数...')
x = eval(text)
```

```
text = input('请输入两个整数，以逗号隔开...')
x, y = eval(text)
```

```
a = eval('4, 5')           # a = 4, 5
a = eval('2 + 3*4')        # a = 2 + 3*4 = 14
a = eval('a + 5')          # a = a + 5 = 19
a = eval('3.14')           # a = 3.14
a = eval('"abc"')          # a = 'abc'
a = eval('aa')              # a = aa, 抛出异常名字aa未定义
a = eval('a = 3')          # 语法错
exec('a=4')                 # a = 4
eval('print("a + 4 =", a+4)') # 输出a + 4 = 8, 返回None
```

```
def eval_demo():
    while True:
        try:
            text = input(">>> ")
            result = eval(text)
            print(type(result), repr(result))
        except InterruptedError:
            break
        except Exception as e:
            print(type(e), e)
```

eval_demo.py

eval函数是不安全的

```
>>> cmd = "__import__('os').startfile(r'C:\\Windows\\notepad.exe')"  
>>> print(cmd)  
__import__('os').startfile(r'C:\\Windows\\notepad.exe')  
>>> eval(cmd)  
>>> eval("__import__('os').system('dir')")
```

eval_unsafe.py

- 上述代码可以看到eval传递的参数来自于用户的输入时，调用eval函数是不安全的！！
- __import__为模块builtins对象（__builtins__）中的内置函数，eval可以传递第二个参数(名字空间)，这样在计算表达式时会查看第二个参数给出的名字空间中的名字

```
>>> safe_dict = {"__builtins__": {}}  
>>> eval("__import__('os').system('dir')", safe_dict)  
...  
NameError: name '__import__' is not defined
```

```
import ast  
try:  
    cmd = input("...")  
    result = ast.literal_eval(cmd)  
except ValueError:  
    traceback.print_exc()
```

字符串对象支持的算术运算符: + * %

- **加法运算符: s1 + s2**

- 字符串与字符串相加, 生成一个新的字符串, 由两个字符串合并而成(**concatenate**)
- 字符串不能与其他类型对象相加

- **乘法运算符: s * n 或 n * s**

- 字符串与整数相乘, 生成一个新的字符串, 相当于字符串的内容重复多次
- 整数小于等于0时生成空字符串 ""

```
>>> a = 'abc' + '123'           #生成新对象 'abc123'
>>> print(a)
abc123
>>> 'abc' + 4
Traceback (most recent call last):
  File "<pyshell#261>", line 1, in <module>
    'abc' + 4
TypeError: Can't convert 'int' object to str implicitly
>>> 'abc' + str(4)
'abc4'
```

```
>>> print('abc' * 4)
abccabccabccabc
>>> print('-' * 25)
-----
>>> print(4 * '*-')
*_*_*_*_
```

- **格式化运算符 %**
format_string % value
format_string % (value1, value2, ...)

字符串比较

- 等于和不等于 (==, !=) 几乎可以用于任何类型, 而且不同类型的对象也可以比较相等关系
 - 相等: 除了数值类型之间的相等比较外, 一般要求类型相同, 且值相等
- 数值类型 (bool、整数、浮点等) 的对象之间可比较大小
- 字符串和字符串可以比较大小: 基于字符之间的大小关系比较
 - 字符之间的比较按照其在Unicode字符集中出现的先后顺序: 空格... 0...9 ...A...Z... a...z ...
 - 两个字符串的大小关系比较: 基于字典顺序
 - 按顺序逐个比较各个字符的大小, 如果前面比较确定了大小关系, 则结束。如果其中某个字符串已经到了结尾, 即没有对应的字符, 则无字符表示其最小
- 算术运算符和关系运算符的优先级? 算术运算符的优先级更高, 即先算再比较

```
>>> 4 != 'ab'
True
>>> '0' < '1'
True
>>> 'a' > 'A'
True
>>> '15' < '17'
True
```

```
>>> '15' < '132'
False
>>> '15' < '152'
True
>>> 'ABC' < 'C' < 'Pascal' < 'Python'
True
>>> 'abc' + '1' < 'abc2'
True
```

成员关系判断运算符 in和not in

判断字符串word是否出现在字符串text中

- word in text 如果word出现在text中, 结果为True, 否则False
- word not in text 如果word没有出现在text中, 结果为True, 否则False

```
>>> import string
>>> dir(string)
[ 'Formatter' , 'Template ',... , 'ascii_letters',
'ascii_lowercase', 'ascii_uppercase', 'capwords', 'digits',
'hexdigits', 'octdigits', 'printable', 'punctuation',
'whitespace']
>>> string.hexdigits
'0123456789abcdefABCDEF'
>>> string.digits
'0123456789'
>>> string.whitespace
' \t\n\r\x0b\x0c'
>>> string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.ascii_lowercase
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

```
>>> text = 'python programming'
>>> 's' in text
False
>>> 's' not in text
True
>>> 'python' in text
True
```

```
>>> ch = input('please input a character: ')
please input a character: 0
>>> '0' <= ch <= '9'
True
>>> ch in string.digits
True
```

额外的Slides

课堂练习

• 下面不是合法的整数字面量的是()

- A. 0x1e B. 1e2 C. 0b1001 D. 0o29 E. 0x-20

下列运算结果不是浮点数的有()

- A. $2 * 2.5$ B. $2 ** -1$ C. $5 // 2$ D. $18 / 3$

表达式 $3 * (2 + 12\%3) ** 3 / 5$ 的结果为()

- A. 129.6 B. 4 C. 43.2 D. 4.8

表达式 $18 / 2 \% 2 * 3 ** 3$ 的运算结果为()

课堂练习

- 写出相应的表达式

$$2^{100} \quad \frac{6}{23} \quad \sin \pi \quad \log_2 1024 \quad 7 \bmod 2 \quad 10x + y$$
$$\sqrt{3412789} \quad \sqrt[3]{x} \quad |-1869| \quad \binom{69}{18} \quad e^{-x} \quad 10!$$

课堂练习

- 采取多种方式（字符串转义、长字符串、原始字符串）定义一个字符串，该字符串应该包含下面两行文字

It's ok.

"Thank you very much!" she said.

- 下面的表达式中会出现错误的是()

A. int('10.8') B. float('10 ') C. int('10')
D. float(10.8) E. int(10.8) F. float('1.08e+1')

Decimal (拓展的内容)

decimal模块支持十进制高精度浮点数Decimal，保证用户指定的精度（缺省28位）

- Decimal(value='0')：创建Decimal对象时，不考虑精度以及舍入(round)等
 - value可以是整数、浮点数和Decimal对象。浮点数在内部表示时已经有了误差
 - 建议value传递十进制整数或浮点数（小数或科学记数法）字符串

```
>>> from decimal import Decimal, getcontext, localcontext
>>> s0 = Decimal('0.1')
>>> s1 = Decimal(0.1)    # Decimal.from_float(0.1)
>>> s0
Decimal('0.1')
>>> s1
Decimal('0.100000000000000000055511151231257827021181583404541015625')
>>> x0 = 1.1
>>> x0 * x0
1.21000000000000002
>>> x1 = Decimal('1.1')
>>> x1 * x1
Decimal('1.21')
>>> Decimal('0.100')
Decimal('0.100')
>>> Decimal('1e-4')
Decimal('0.0001')
```

Decimal (拓展的内容)

上下文：给出了Decimal对象进行运算时保留的有效数字以及遵循的舍入规则等

- 内部表示：符号sign + 系数coefficient（整数） + 指数 exponent（整数） $-1^{\text{sign}} \times \text{coefficient} \times 10^{\text{exponent}}$
 - d.as_tuple() 查看内部表示
- prec: 精度，有效数字的个数，即系数部分的数字的个数
- round: 计算结果的系数部分长度超过了prec时，舍去部分数字时确定如何舍入，缺省四舍六入五成双
- 一般建议首先设置更大的prec，在得到最终结果后再round到更小的prec

```
>>> from decimal import Decimal, getcontext, localcontext
>>> d1 = Decimal('0.01230')
>>> d1.as_tuple()
DecimalTuple(sign=0, digits=(1, 2, 3, 0), exponent=-5)
>>> ctx = getcontext()
>>> ctx
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=[],
traps=[InvalidOperation, DivisionByZero, Overflow])
>>> ctx.prec = 4
>>> Decimal('12345') + Decimal('10')
Decimal('1.236E+4')
>>> _.as_tuple()
DecimalTuple(sign=0, digits=(1, 2, 3, 6), exponent=1)
```

Decimal (拓展的内容)

- `normalize()` 规格化, 去除系数部分最右边的0
- `quantize(exp)` 用于指定保留小数点后多少位。返回一个相同的值, 该值的指数与`exp`的指数一样
 - 关键是`exp`的小数点之后的值
- 如果只是临时设置上下文执行一段代码, 使用`localcontext + with`语句

```
>>> from decimal import Decimal, getcontext, localcontext
>>> Decimal('12300').normalize()
Decimal('1.23E+4')
>>> Decimal('0.45600').normalize()
Decimal('0.456')
>>> Decimal("2.3456").quantize(Decimal("0.01"))
Decimal('2.35')
>>> Decimal("1") / Decimal("7")
Decimal('0.1428571428571428571428571429')
>>> with localcontext(prec=4):
...     print(Decimal("1") / Decimal("7")) # 0.1429
```

```
with localcontext(prec=4):
    # code block
```

- 保存当前上下文
- 基于当前上下文创建临时上下文, 设置其精度=4
- 执行code block
- 执行完之后恢复之前保存的上下文